Rheinisch-Westfälische Technische Hochschule Aachen
Alessandro Berti

Phase Review: 1st Sprint of the Implementation of "Conformance Predictions"

Group 4:
Richard Schuppener, Cem Colak

Contents

1 Summary of Sprint 1 **3**

2 Requirements Coverage Status - Overview per Module **3**
2.1 Running Highlevel Requirements 3
2.2 IO 3
2.3 Feature Extraction 4
2.4 Model Training 5
2.5 Evaluation 5
2.6 Config & Pipeline 6
2.7 Project Infrastructure & Testing 7

3 User Guide **8**

4 Team settings and Workflow **8**
4.1 Changes in Responsibilities 8
4.2 Workflow Improvements for the next sprint 8

5 Exemplary Visualisations and Outputs from the Tool **9**

6 Sprint review **9**
6.1 Richard Schuppener 9
6.2 Cem Colak 9

1 Summary of Sprint 1

Achievements of Sprint 1 (based on GitLab work-items) from high level perspective, short and no technical details

2 Requirements Coverage Status - Overview per Module

2.1 Running Highlevel Requirements

Add a high-level status overview of the top-level HL requirements across all modules (e.g., as a table with status: covered / partially covered / planned).

2.2 IO

Covered

- **LL-01.1**: `io.discover_process_model()` supports Inductive, Alpha, and Genetic Miner via PM4Py, enabling automatic model discovery when no normative model is available.
- **LL-01.2**: `io.visualize_process_model()` renders the model as Petri net, BPMN, or Process Tree, with optional file saving or direct display.
- **HL-05**: Event log loading, model import, and model discovery all functional; the foundational data acquisition step of the pipeline is complete.
- **LL-05.1**: `pm4py.read_xes` for XES logs; `pd.read_csv` combined with `pm4py.format_dataframe` for CSV logs in `io.load_event_log()`.
- **LL-05.2** (partially): Petri net loading via `pm4py.read_pnml` implemented; BPMN files loaded via `pm4py.read_bpmn` and internally converted to a Petri net.
- **LL-05.3**: Inductive, Alpha, and Genetic Miner available as fallback discovery algorithms when no normative model file is provided.
- **HL-04**: `try/except` blocks in `main.py` catch and report setup errors, preprocessing failures, and pipeline exceptions to `stderr` with descriptive labels.

Planned in Sprint 2

- **HL-01**: End-to-end prediction of conformance deviations as a fully operational business deliverable not yet achieved; requires conformance deviation prediction target to be fully wired through the complete pipeline.
- **LL-05.2**: Standalone loading of BPMN and Process Tree model files as direct model inputs (without forced internal conversion).

2.3 Feature Extraction

Covered

- **HL-06** (partially): `feature_extraction.compute_alignments()` invokes `pm4py.conformance_diagnostics_alignments`; log-moves, model-moves, and synchronous moves extracted per case. Distance-to-completion indicators not yet included.
- **LL-06.1**: PM4Py alignment dictionaries computed for every trace against the normative Petri net using the standard cost function.
- **LL-06.2**: Alignment moves parsed in `extract_label()` to detect and count deviations (log-skips and model-skips) per case; cumulative alignment cost extracted.
- **HL-07** (partially): `feature_extraction.extract_features()` supports control-flow features (Bag-of-Activities), alignment-based diagnostic features (Bag-of-move-types per activity), and a hybrid combination. Case attribute extraction is defined in the code but commented out.
- **LL-07.3**: Control-flow frequency matrix constructed via `pd.crosstab` over case IDs and activity names, producing a per-case Bag-of-Activities feature matrix.
- **HL-13**: Alignment results computed once per run and stored in-memory; passed as a variable to avoid redundant re-computation within a single pipeline execution.

Planned in Sprint 2

- **HL-02**: Full integration of structured case attributes (e.g., claimant demographics, claim amount) as predictive features alongside process-level signals.

- **HL-06**: Addition of distance-to-completion indicators (e.g., remaining enabled transitions, token distance to final marking).
- **LL-07.1**: Prefix generation loop that slices event sequences to varying lengths k for prefix-level prediction.
- **LL-07.2**: Numerical feature matrix construction tailored to remaining time regression targets.
- **LL-13.1**: Hash-based caching of prefix activity sequences to deterministically identify and skip redundant alignment computations.
- **LL-13.2**: Disk-based persistence of alignment results via `joblib`.

2.4 Model Training

Covered

- **HL-08** (partially): `model_training.create_model_pipeline()` wraps a Decision Tree classifier in a `sklearn.Pipeline`. Only one algorithm (Decision Tree) currently supported; final alignment cost as deviation threshold not yet used.
- **LL-08.1**: `StandardScaler` integrated as the first step in the sklearn Pipeline, ensuring scale-consistent feature distributions.
- **HL-09**: Baseline (control-flow features only) and conformance-aware (diagnostics or hybrid features) model variants both supported via the `PredictionFeature` configuration.

Planned in Sprint 2

- **HL-08**: Use of case final alignment cost as the deviation threshold for defining the binary classification target.
- **LL-08.2**: Additional ML algorithms beyond the Decision Tree (e.g., Random Forest, Logistic Regression) for the controlled comparison.

2.5 Evaluation

Covered

- **HL-15** (partially): `evaluation.temporal_split()` implemented, partitioning cases by end-time into train/val/test sets. The main pipeline currently uses a simplified `pm4py.split_train_test` 75/25 split rather than the full temporal split.

- **HL-16** (partially): `evaluation.compute_classification_metrics()` computes accuracy, precision, recall, F1-score, ROC-AUC, and Brier score for binary and multi-class classification targets.
- **HL-12**: Confusion matrix heatmap and per-class accuracy bar chart generated via Matplotlib in `evaluation.save_evaluation_visuals()`.

Planned in Sprint 2

- **HL-15**: Full integration of `evaluation.temporal_split()` into the main pipeline; enforcement of strict temporal case ordering with train, validation, and test sets.
- **HL-16**: Conformance quality metrics for the normative model (fitness, precision, generalization, simplicity).
- **LL-16.2**: MAE and RMSE evaluation metrics for remaining time regression targets.
- **HL-17**: Prefix-length performance analysis using `evaluation.evaluate_by_prefix_lengt` (function stub already present).

2.6 Config & Pipeline

Covered

- **HL-03** (partially): `main.py` provides an interactive CLI guiding the user through model path/discovery selection, prediction target (future deviations, process outcome, remaining time), feature basis, and ML algorithm configuration. Conformance deviations as target and case attributes as feature basis not fully functional end-to-end.
- **LL-03.1** (partially): Prediction target and feature basis options fully defined in `config.py` and selectable in the CLI; case attribute extraction and conformance deviation target not yet fully wired through the pipeline.
- **HL-10**: `main.py` combined with `pipeline.py` forms an end-to-end runner from raw data and model input to evaluation output, without manual intermediate steps.

Planned in Sprint 2

- **HL-03**: Full CLI support for conformance deviations as prediction target and case attributes as feature basis.

- **HL-14**: Runtime and resource measurement wrapping conformance checking and feature generation steps.
- **LL-14.1**: `time.perf_counter()` timing instrumented around alignment computation and model training calls.
- **LL-14.2**: `psutil.Process().memory_info()` peak RAM tracking during preprocessing and alignment.
- **LL-14.3**: Millisecond-level start/end timestamps recorded per batch for the combined conformance checking and feature generation steps.
- **LL-14.4**: Runtime and resource metrics printed to the console and persisted in a structured log file (CSV or JSON).

2.7 Project Infrastructure & Testing

Covered

- **HL-18**: PM4Py, scikit-learn, pandas, NumPy, and Matplotlib used as the primary library stack throughout all modules.
- **HL-19 & LL-19.1-19.2**: Project organized into `io`, `feature_extraction`, `model_training`, `evaluation`, `pipeline`, and centralized `config` modules; Git used for version control. Note: conformance logic currently consolidated into `feature_extraction` rather than a dedicated `conformance` module. Branching strategy for each module not considered, as we are working on two branches for each team member.
- **HL-20**: `requirements.txt` and `README.md` present in the repository.
- **LL-20.1**: `requirements.txt` as a plain text file implemented and considered satisfactory for dependency management so far.

Planned in Sprint 2

- **HL-11**: Automated pytest test suite executable via a single command.
- **LL-11.1**: Pytest fixtures providing a synthetic, fully conforming XES log and corresponding Petri net.
- **LL-11.2**: Test assertion that alignment cost equals 0.0 on fully conforming synthetic data.
- **LL-11.3**: Test assertion that a prefix of length k contains strictly only the first k events of the original trace.

- **HL-21**: Technical documentation of system architecture, data and model flow, conformance checking strategy, feature design, and main implementation choices. An architecture diagram is considered as a key component of this documentation.

3 User Guide

To faciliate the usage of our tool, we have created a user guide, which provides the user with step-by-step instructions on how to run the tool and what its limits are. For an end-user and tester of our tool, the user guide is an essential resource to really see the functionality of our software artifact. We will update the user guide after each sprint to reflect the current state of the tool. In the submissions for each sprint, we will include the user guide as a separate document, which is also available in our GitLab repository. User can find it under each sprint folder, e.g. `Sprint 1/UserGuide.pdf`.

4 Team settings and Workflow

4.1 Changes in Responsibilities

For the next sprint, we will switch some of the responsibilites for the modules to enhance boths understanding for each module. This is still under discussion and we will decide on the final allocation of the responsibilities for the next sprint in the next call.

4.2 Workflow Improvements for the next sprint

In GitLab we used the work items to consolidate the requirements, such that we have an overview of all the requirements for specific modules. That helped us a lot for the first sprint, to not lose track of the overall picture of the modules. For the next sprints we planned to improve the workflow by formulating them in a more granular and specific manner. This strategy arises by the fact, that we now want to focus more on specific requirements and track the progress in a clear and traceable manner.

Furthermore, we will maintain the breakdown of the requirements per module listed here in the review document and update it after each sprint to keep track of the coverage status. This way, we can easily identify which requirements are still open and need to be addressed in the next sprints.

5 Exemplary Visualisations and Outputs from the Tool

Exemplary visualizations (from IO, Evaluation module) and numerical results e.g. accuracy, class accuracy (from Evaluation module)

6 Sprint review

6.1 Richard Schuppener

6.2 Cem Colak

This phase was the most challenging to me, since I had no prior experience with building software artifacts. Especially the dependencies of the different modules, the connection to the CLI and the pipeline were completely new to me. This bottleneck was very well handled by Richard, who set up the code infrastructure and explained me the dependencies within the modularity, such that I was able to work properly on my modules. Towards the end of the second week, I noticed a huge learning curve regarding my understanding of the dependecies of each module, because I became able to implement more options into the CLI (e.g. the choice of the Algorithms) and embedded it into the whole pipeline. Our communication was seamless, especially because we both have some lectures in common and used the time after the lectures to work on our project. Despite that, we have a regular call each Friday morning where we discuss next steps and prepare our work allocation for the weekend. On Friday the 16th we talked about the status of our work and our time contstaints during the incoming weekend. I was very happy that Richard suggested to ask Alessandro if we could postpone the submission by one day - giving us more time to deliver quality work. In the next sprint, I will travel in the second week. This could impact and challenge our progress, but I am confident that we can adapt and manage our timetable accordingly. I am very satisfied and thankful with Richards contribution and support in this phase, hence I am looking forward to the next sprint, where we will be able to build more functionalities on the strong foundation we have build in this sprint.